University of Jaffna

Department of Computer Science

Impact of Monitoring Baselines on Microservice Failure Detection

Research Project Proposal

2022/CSC/093

Submitted by: W.L.D.P.U.D.A. Goonatilake
Supervisor: Dr. (Ms.) J. Samantha Tharani
Date: 15 October, 2026

Contents

Abstract **1**

1 Introduction **1**
1.1 Background of the Study 1
1.2 Problem Statement and Research Gap 2
1.3 Research Aim 2
1.4 Research Objectives 2
1.5 Research Questions 3

2 Literature Review **3**
2.1 Overview of the Research Area 3
2.2 Review of Existing Studies 3
2.3 Research Gap 4

3 Research Methodology **4**
3.1 Research Design 4
3.2 Dataset 5
3.3 Proposed Approach 5
3.4 Implementation and Tools 7
3.5 Evaluation and Analysis 8
3.6 Ethical Considerations 8

4 Expected Outcomes **9**

5 Research Timeline **9**

References **10**

Abstract

Cloud-native applications often depend on many connected services, so a problem in one service can affect the whole application. Microservice monitoring helps identify such problems by observing the computing resources used by services and how well they respond to requests. However, normal system behaviour may differ between experimental runs. As a result, a failure detector that learns what is normal from one run may raise unnecessary alerts or miss failures in another.

This research examines how the choice of normal monitoring data influences microservice failure detection. It will compare normal data from the same experimental run, from a different run, and from several other runs combined. Under each condition, the study will consider three groups of monitoring information: infrastructure metrics, service response signals, and their combination. The evaluation will use RCAEval [1], a public dataset containing measurements from microservice applications during controlled fault-injection experiments.

The study will assess how reliably failures are identified, how often normal behaviour is incorrectly reported as a failure, and whether the results differ across fault types and applications. The expected outcome is a clearer understanding of how baseline selection affects monitoring reliability and whether conclusions about the usefulness of different monitoring signals remain consistent across runs.

1 Introduction

1.1 Background of the Study

Cloud-native microservice applications are composed of interconnected services. Their behaviour is monitored using internal resource measurements and service-level request measurements. CPU, memory, disk and socket metrics describe infrastructure activity; latency, request errors and throughput provide information about service-level request behaviour. Existing research shows that different monitoring data can support anomaly detection, either separately or together [2, 3].

Anomaly detectors commonly learn from observations assumed to represent normal operation. Their results can therefore depend on which normal observations are available during training. If normal resource usage or response latency varies between experimental runs, a previously fitted detector may raise false alarms or miss injected faults. Such differences are relevant to Site Reliability Engineering because monitoring needs to remain useful beyond a single training period.

1.2 Problem Statement and Research Gap

Research has established anomaly detection using profiling measurements and response-time features [2, 3]. ReplicaWatcher also addresses problems caused by changing normal behaviour, although it proposes a training-less method primarily for security-related anomalies [4]. Consequently, neither monitoring-signal combination nor baseline reliability can be claimed as new.

However, the studies reviewed provide limited direct evidence on whether the source of normal training data affects the failure-detection effectiveness of infrastructure-only, service-response-only and combined monitoring differently. This study will compare normal data from the same experimental run, a different run and several other runs combined, while keeping the detection method and held-out test observations consistent. The proposed contribution is a controlled empirical comparison of these effects rather than a new failure-detection method.

1.3 Research Aim

The aim of this study is to investigate how the selection of normal monitoring baselines influences the effectiveness of microservice failure detection using infrastructure metrics, service response signals and their combination.

1.4 Research Objectives

The specific objectives are:

1. Review relevant microservice failure-detection and normal-baseline research.
2. Verify usable normal periods, repeated runs, fault annotations and metric consistency in RCAEval [1].
3. Construct three monitoring configurations and compare three normal-baseline sources under one detection procedure.
4. Measure detection reliability on held-out observations and analyse differences across fault types and applications.

1.5 Research Questions

RQ1: How does the source of normal monitoring data affect the effectiveness of microservice failure detection using infrastructure metrics, service response signals and their combination?

RQ2: How does the effect of baseline selection vary across microservice fault types and applications?

2 Literature Review

2.1 Overview of the Research Area

Failure detection identifies potentially abnormal system behaviour, while root-cause analysis seeks to determine the source of a failure. Microservice failure detection can use infrastructure measurements, service-level request measurements, or their combination. Detection methods that learn from normal observations may be affected by changes in normal system behaviour. Therefore, both the monitoring information used and the source of normal training data are relevant to detection reliability.

2.2 Review of Existing Studies

Panahandeh *et al.* propose ServiceAnomaly, which combines distributed traces and profiling metrics to identify abnormal behaviour in microservices [2]. Akmeemana *et al.* present GAL-MAD, which uses performance metrics and response-time measurements in a graph-based anomaly detection approach [3]. These studies demonstrate the relevance of different monitoring measurements for anomaly detection, but their reported evaluations do not directly compare the three normal-baseline sources proposed in this research.

El Khairi *et al.* introduce ReplicaWatcher, a training-less approach that compares container replicas to detect security-related anomalies under changing operating conditions [4]. Their work highlights the limitations of relying on historical normal behaviour, but it does not compare normal training data from different experimental runs for operational failure detection.

Pham *et al.* introduce RCAEval, a public benchmark developed primarily for microservice root-cause analysis [1]. Its monitoring measurements and fault-injection records provide a basis for the failure-detection experiments proposed here. Liu *et al.* introduce Isolation Forest, a lightweight anomaly detection method that can be used consistently across the proposed conditions [5]. The summary of literature review shown in Table 1.

Table 1: Selected related studies and relevance to the proposed research

Study	Main focus	Difference from this study
ServiceAnomaly [2]	Anomaly detection using traces and profiling metrics	Does not compare the proposed normal-baseline sources.
GAL-MAD [3]	Graph-based detection using performance and response-time measurements	Focuses on the detection approach rather than baseline-source effects.
ReplicaWatcher [4]	Training-less detection under changing normal behaviour	Uses replica comparison in a security-focused setting.
RCAEval [1]	Public benchmark for microservice root-cause analysis	Provides experimental data rather than the proposed baseline comparison.

2.3 Research Gap

Existing studies demonstrate the usefulness of different monitoring measurements for anomaly detection and recognise challenges caused by changing normal behaviour. However, the reviewed literature provides limited direct evidence on whether the source of normal training data affects infrastructure-only, service-response-only and combined monitoring differently under consistent fault-injection conditions.

This research aims to provide a controlled empirical comparison of these effects by evaluating detection performance, false alarms and differences across fault types and applications. The proposed contribution is experimental evidence about baseline selection rather than a new detection method. The research gap will continue to be assessed against relevant literature.

3 Research Methodology

3.1 Research Design

This is a quantitative empirical study with two factors: monitoring input group and normal-baseline source. The three monitoring groups are infrastructure-only (A), service-response-only (B) and combined (C). The baseline conditions are same-run, other-run and pooled-run, shown in Table 2. This yields nine combinations, evaluated using the same held-out target observations within each experiment. Isolation Forest will be fitted separately for each combination; its detector settings and threshold-selection procedure will be held consistent.

Table 2: Three monitoring configurations under three normal-baseline conditions

Monitoring configuration	**Same-run**	**Other-run**	**Pooled-run**
A: Infrastructure metrics	Evaluate	Evaluate	Evaluate
B: Service response signals	Evaluate	Evaluate	Evaluate
C: Combined signals	Evaluate	Evaluate	Evaluate

3.2 Dataset

The study will use the public **RCAEval** dataset, released with the benchmark paper [1]. It contains 735 fault-injection cases across Online Boutique, Sock Shop and Train Ticket, organised into RE1, RE2 and RE3. The experiments include repeated fault injections and metric time series, allowing selected normal periods to be compared across runs.

The project will use only metric tables and fault-injection metadata. Infrastructure features may include CPU, memory, disk I/O and socket measurements. Response-related features may include service latency, error indicators and workload or request-rate measures, depending on verified column definitions. These are aggregated *service-level metrics*, not necessarily individual client-side API responses. Logs and traces are outside the experimental scope.

A data audit will verify normal-period duration, timestamp validity, feature definitions, repetition compatibility and fault types. Earlier feasibility work suggests that some repeated normal periods differ, but those preliminary results will be independently checked. The audit will identify cases with sufficient normal observations, compatible features and enough repetitions for the proposed comparisons. Only eligible cases will be included, and exclusion rules will be documented. Fault-injection timestamps will be used as reference points, not as evidence of the exact duration of service degradation.

3.3 Proposed Approach

Phase 1: Prepare monitoring inputs.

Group documented features into A, B and C. Select a common set of columns for comparable runs of the same application. Record missing features and avoid selecting inputs using test outcomes.

Phase 2: Define baseline sources.

1. **Same-run:** Fit on an earlier, pre-injection normal portion of the target run, leaving separate normal observations for validation and testing.
2. **Other-run:** Fit on normal observations from one other repetition of the same application and fault–service scenario.
3. **Pooled-run:** Fit on normal observations drawn from multiple other eligible repetitions; exclude the target run from this baseline.

Equal training sample sizes and fixed selection rules will be used where possible. Same-run training has access to target-specific history, so it represents a different operational condition from the two donor-run baselines. This distinction will be reported rather than treated as a perfectly controlled difference in data source alone.

Phase 3: Separate training, calibration and testing.

Keep these sets disjoint. Normal target observations reserved for false-alarm testing will not also be used to select thresholds. Donor-run baselines will be calibrated on their own independent normal validation observations. All nine conditions will be tested on matching held-out target observations. Imputation and scaling rules, if needed, will be learned from training data only.

Phase 4: Detect failures.

Fit Isolation Forest to the assigned normal data [5]. Choose thresholds with a predefined validation procedure, without using test fault prevalence. Apply the models to the same target normal test portion and a predefined post-injection observation interval. Rotate target repetitions where feasible.

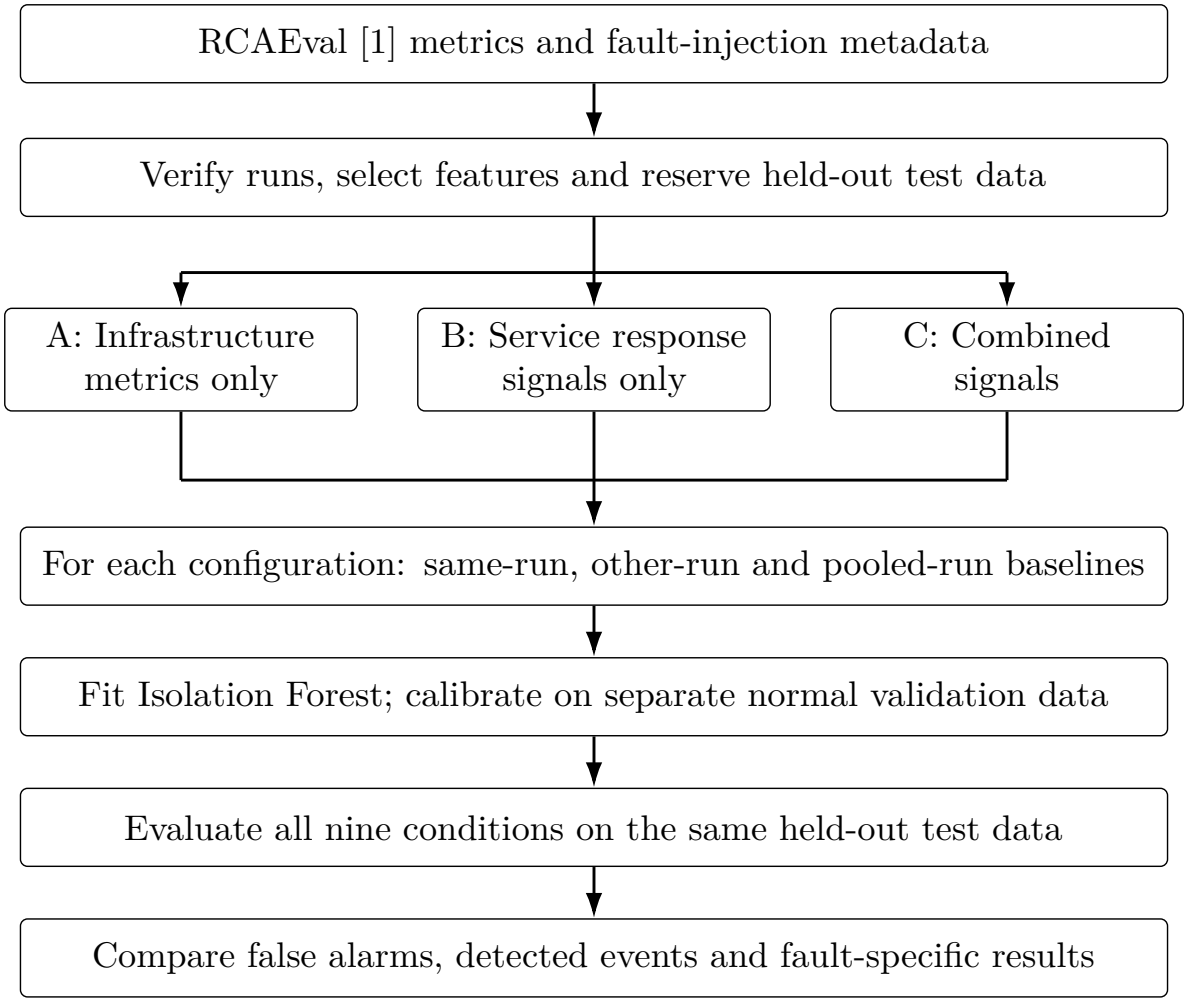


Figure 1: Proposed research methodology.

3.4 Implementation and Tools

Experiments will use Python, Pandas and NumPy for data preparation, scikit-learn for Isolation Forest, and Matplotlib for visualisation. The public RCAEval dataset [1] will be analysed locally, without requiring a new Kubernetes deployment or deep-learning system. Scripts and experimental settings will be documented to support reproducibility.

3.5 Evaluation and Analysis

The main evaluation measures will be the false-positive rate (FPR) on held-out normal observations, the proportion of fault-injection events associated with at least one detection within a predefined time window, and the time from injection to the first detection. The observation window will be fixed before examining test results. Precision, recall and F1-score may be reported as supporting measures, with the limitations of injection-based labels clearly stated.

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{1}$$

$$F_1 = \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}} \qquad \text{FPR} = \frac{FP}{FP + TN} \tag{2}$$

Here, TP, FP, TN and FN denote true positives, false positives, true negatives and false negatives under the explicitly stated labelling policy. Undefined ratios will be reported as undefined. Because some faults may stop or recover after injection, post-injection observations will not automatically be treated as a fully verified continuous outage.

The main analysis will compare *changes in detection results between baseline sources* for each input group. It will also examine whether relative configuration rankings change across resource, network/communication and code-level faults. Uncertainty will be estimated across independent experiment runs rather than by treating adjacent one-second measurements as independent replicates. No baseline strategy or monitoring group is assumed to perform best.

3.6 Ethical Considerations

The study uses a publicly available benchmark and does not involve human participants or collect personal information. Dataset usage and attribution requirements will be followed. Experimental exclusions, uncertain labels and limitations will be reported transparently.

The main validity risks include differences in information available to baseline conditions, inconsistent monitoring features across runs, and the use of controlled rather than production faults. These limitations will be considered when interpreting the results.

4 Expected Outcomes

The study is expected to produce a documented dataset-preparation and baseline-selection procedure, along with a consistent evaluation of failure detection across nine experimental conditions. The results will provide evidence on whether changing the source of normal training data affects false alarms and failure detection, and how these effects vary across monitoring configurations, fault types and applications.

The main contribution will be empirical evidence on the effects of baseline selection in microservice failure detection. The findings may identify monitoring configurations that perform consistently across different baselines, or show that their effectiveness depends on the application and fault type. These results will help clarify how baseline selection influences the interpretation of failure-detection performance.

5 Research Timeline

The proposed 12-month research plan is presented in Table 3. The activities cover literature review, dataset preparation, experimental evaluation, analysis and final dissertation submission.

Table 3: Proposed Research Timeline (12 Months)

Research Activity	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**	**M9**	**M10**	**M11**	**M12**
Literature review and gap refinement	X	X	X									
Proposal and supervisor feedback	X											
RCAEval audit and case selection	X	X	X									
Baseline variation pilot		X	X	X								
Feature preparation and evaluation design			X	X	X							
Implementation of all conditions				X	X	X						
Full experiments and initial analysis						X	X	X				
Fault-type and application analysis							X	X	X	X		
Robustness checks and interpretation									X	X	X	
Research documentation and supervisor feedback	X	X	X	X	X	X	X	X	X	X	X	X
Dissertation writing and revision				X	X	X	X	X	X	X	X	X
Publication preparation and submission									X	X	X	X
Final dissertation and submission											X	X

References

[1] L. Pham, H. Zhang, H. Ha, F. D. Salim, and X. Zhang, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in *Companion Proceedings of the ACM Web Conference 2025*. ACM, 2025, pp. 777–780. [Online]. Available: https://doi.org/10.1145/3701716.3715290

[2] M. Panahandeh, A. Hamou-Lhadj, M. Hamdaqa, and J. Miller, "ServiceAnomaly: An anomaly detection approach in microservices using distributed traces and profiling metrics," *Journal of Systems and Software*, vol. 209, p. 111917, 2024. [Online]. Available: https://doi.org/10.1016/j.jss.2023.111917

[3] L. Akmeemana, C. Attanayake, H. Faiz, and S. Wickramanayake, "GAL-MAD: Towards explainable anomaly detection in microservice applications using graph attention networks," in *Proceedings of the 2026 Australasian Computer Science Week (ACSW)*. ACM, 2026, pp. 11–21. [Online]. Available: https://doi.org/10.1145/3793811.3793819

[4] A. E. Khairi, M. Caselli, A. Peter, and A. Continella, "ReplicaWatcher: Training-less anomaly detection in containerized microservices," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024. [Online]. Available: https://doi.org/10.14722/ndss.2024.24286

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422. [Online]. Available: https://doi.org/10.1109/ICDM.2008.17